

Kernelarchitektur

Wie generischer Kernelcode, Subsysteme und Hardware zusammenwirken

1.04

Vier Ebenen strukturieren den Weg zur Hardware

User Space

Anwendungen · Bibliotheken · Laufzeitumgebungen

System Calls

Kontrollierter Eintritt in privilegierten Code

Kernel Space

Scheduler · MM · VFS · Netzwerk · Treiber

Hardware

CPU · RAM · Controller · Geräte

Linux ist monolithisch – und modular erweiterbar

01 Ein gemeinsamer Kernel-Adressraum

Kernel-Subsysteme und Treiber arbeiten privilegiert und können direkt miteinander interagieren.

02 Klare interne Subsysteme

Scheduler, Speicherverwaltung, VFS und Netzwerk besitzen spezialisierte Schnittstellen und Zuständigkeiten.

MONOLITHISCH

Hohe Integration
kurze interne Wege

Built-in

Module

Module erweitern den Kernel zur Laufzeit – sie bilden aber keine Sicherheitsgrenze.

Generischer Code und arch/<arch> teilen sich die Arbeit

GENERISCH

01 Kernlogik

Scheduler, VFS, Speicher- und Netzwerklogik sind weitgehend architekturübergreifend.

02 Gemeinsame APIs

Subsysteme und Treiber nutzen definierte Kernel-Schnittstellen.

ARCHITEKTURSPECIFISCH

- Exception- und Syscall-Einstieg
- MMU und Page-Table-Details
- Context-Switch-Mechanik
- IRQ- und CPU-spezifischer Code

Portabilität entsteht, weil gemeinsame Logik auf architekturspezifischen Primitiven aufsetzt.

Der Quellbaum spiegelt die Architektur

WICHTIGE VERZEICHNISSE

arch/	CPU- und Plattformcode
kernel/	Kernlogik, Prozesse, Zeit
mm/	Speicherverwaltung
fs/	VFS und Dateisysteme
net/	Netzwerk-Stack
drivers/	Gerätetreiber
include/	gemeinsame Header

01 Vom Symptom zum Subsystem

Ein Dateipfad führt meist zu fs/,
ein Speicherfehler zu mm/,
ein Gerätefehler zu drivers/.

02 Dann zur Architektur

CPU-Eintritt, Page Tables oder Context Switch führen
zusätzlich nach arch/<arch>/.

Ein System Call durchläuft definierte Übergänge

01	User Space	<code>read(fd, buf, len)</code>
02	Arch-Einstieg	Register sichern · Modus wechseln
03	Kernel-Implementierung	Parameter prüfen · Subsystem aufrufen
04	Rückkehr	Ergebnis oder Fehlercode zurückgeben

Die Eintrittsmechanik ist CPU-spezifisch; die fachliche Arbeit liegt im Kernel-Subsystem.

Tasks verbinden Prozessmodell und Scheduler

01 task_struct

Die zentrale Task-Repräsentation bündelt Zustand, Credentials, Speicherbezug und Scheduling-Informationen.

02 Runqueues und Klassen

Der Scheduler hält ausführbare Tasks bereit und wählt gemäß Scheduling-Klasse und Priorität.

AUSFÜHRUNGSKONTEXT

Prozesskontext

→ darf häufig schlafen

Interruptkontext

→ stark eingeschränkt

Bei jeder Kernelanalyse zuerst fragen: In welchem Kontext läuft dieser Code?

Die Speicherarchitektur verbindet Isolation und Effizienz

01 Virtuelle Adressräume

Jeder Prozess erhält eine eigene Sicht; Page Tables übersetzen virtuelle in physische Adressen.

03 Page Cache und Reclaim

Dateidaten können im RAM liegen; unter Druck gewinnt der Kernel Seiten zurück.

02 Allokatoren

Page Allocator und Objekt-Allocator stellen Speicher in unterschiedlichen Granularitäten bereit.

04 Faults und Schutz

Page Faults laden oder erzeugen Abbildungen; Rechte begrenzen Lesen, Schreiben und Ausführen.

Memory Management ist zugleich Abstraktion, Schutzmechanismus und Performance-Schicht.

Das VFS trennt Dateioperationen von Dateisystemen

open() · read() · write()

VFS-OBJEKTE

superblock

inode

dentry

file

ext4

XFS

tmpfs

procfs

weitere

Ein einheitliches Objekt- und Operationsmodell macht unterschiedliche Dateisysteme austauschbar.

Das Linux Driver Model ordnet Busse, Geräte und Treiber

01 Bus

Beschreibt die Verbindung und findet passende Geräte-Treiber-Kombinationen.

02 Device und Driver

Geräte liefern Identität und Ressourcen; Treiber implementieren Verhalten und Lebenszyklus.

BINDING

match → **probe** → **aktiv**

remove → **lösen**

sysfs macht das Modell sichtbar

Das Modell vereinheitlicht Erkennung, Bindung, Lebenszyklus und Sichtbarkeit von Geräten.

Interrupts trennen schnelle Reaktion von späterer Arbeit

01 Exception oder IRQ

Die CPU wechselt über architekturspezifischen Entry-Code in den Kernel.

02 Kurze unmittelbare Behandlung

Der Handler bestätigt die Quelle und sichert nur den nötigen Zustand.

MÖGLICHE FORTSETZUNG

Threaded IRQ

Softirq / NAPI

Workqueue

Der genaue Pfad hängt vom Subsystem ab – nicht jeder Interrupt nutzt dieselbe Deferred-Work-Form.

Konfiguration entscheidet: eingebaut oder Modul

01 Kconfig

Abhängigkeiten und Auswahloptionen bestimmen, welche Funktionen in einen Kernel-Build gelangen.

03 Loadable Module

Module können passende Funktionen nachladen und nutzen exportierte Symbole.

02 Built-in

Fest einkompilierter Code ist bereits beim Start vorhanden.

04 Keine Isolation

Fehlerhafter Modulcode kann den gesamten Kernel beeinträchtigen.

Modularität erleichtert Anpassung und Wartung – sie reduziert nicht die Privilegien des Codes.

Beispiel: read() durchquert mehrere Subsysteme

- | | | |
|----|-------------------|--|
| 01 | Syscall | fd, Buffer und Länge prüfen |
| 02 | VFS | File-Objekt und Operation bestimmen |
| 03 | Page Cache | Daten direkt liefern – falls vorhanden |
| 04 | I/O-Pfad | Bei Cache-Miss: Dateisystem, Blocklayer, Treiber |
| 05 | Completion | IRQ/Completion beendet I/O; Daten zurückgeben |

Ein Cache-Treffer verkürzt den Pfad; Geräte-I/O entsteht nur bei Bedarf.

Kernelarchitektur lesen: drei Leitfragen

01 Welcher Kontext?

Prozesskontext, Interruptkontext oder aufgeschobene Arbeit bestimmen zulässige Operationen.

02 Welches Subsystem?

Scheduler, MM, VFS, Netzwerk oder Driver Core geben Datenmodelle und Schnittstellen vor.

03 Generisch oder architekturenspezifisch?

Die Antwort zeigt, ob der relevante Code im gemeinsamen Kern oder unter arch/<arch>/ liegt.

Architekturkenntnis verkürzt den Weg von einem Symptom zur richtigen Code- und Datenstruktur.